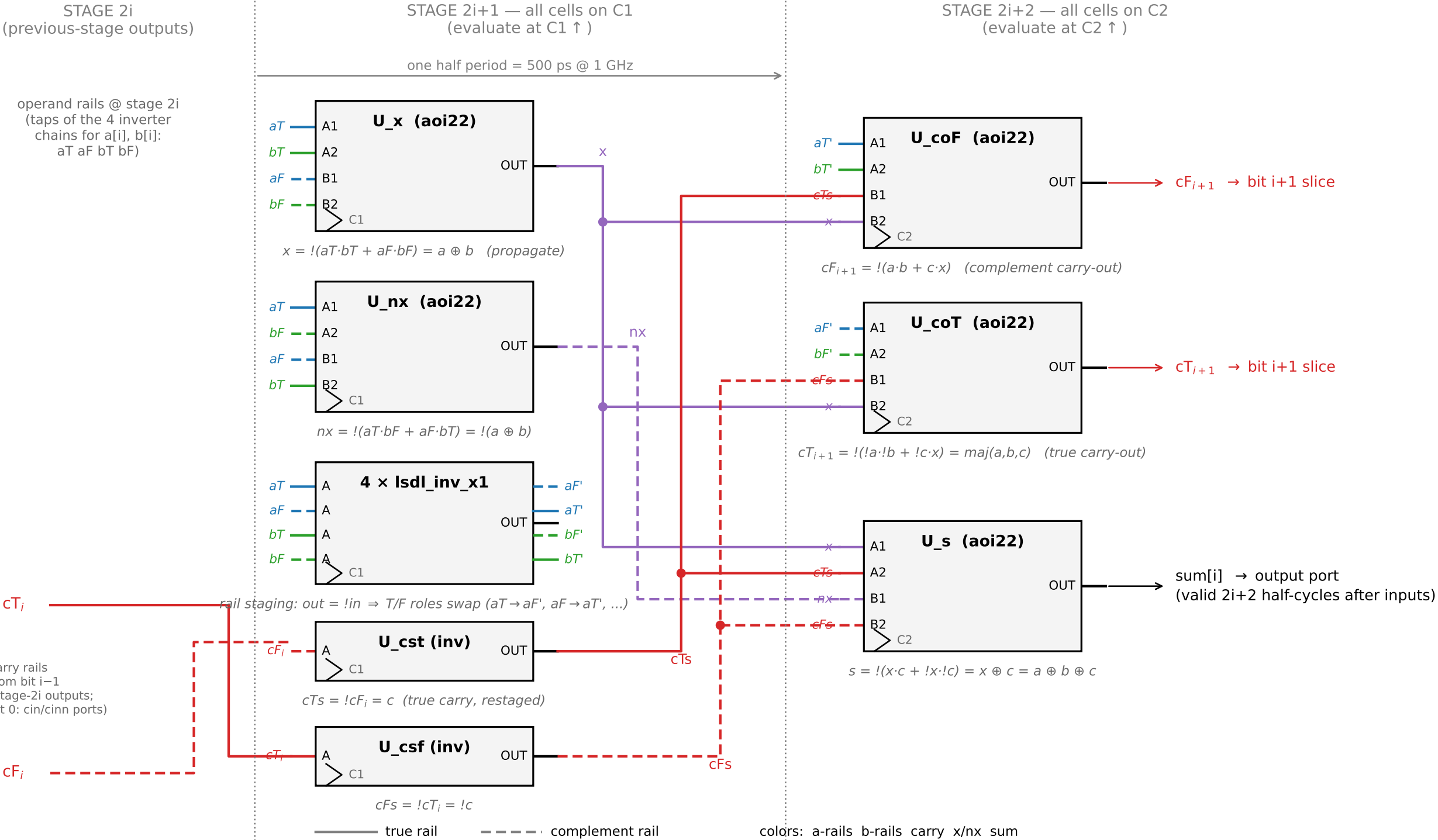


LSDL adder — bit-slice *i* (dual-rail pipelined ripple-carry, gen_lsdl_adder.py)



Every box is an LSDL flop: samples inputs at its CLK rising edge, OUT = !(n-tree function); value latched through the following precharge.

Signal glossary & full-adder identities

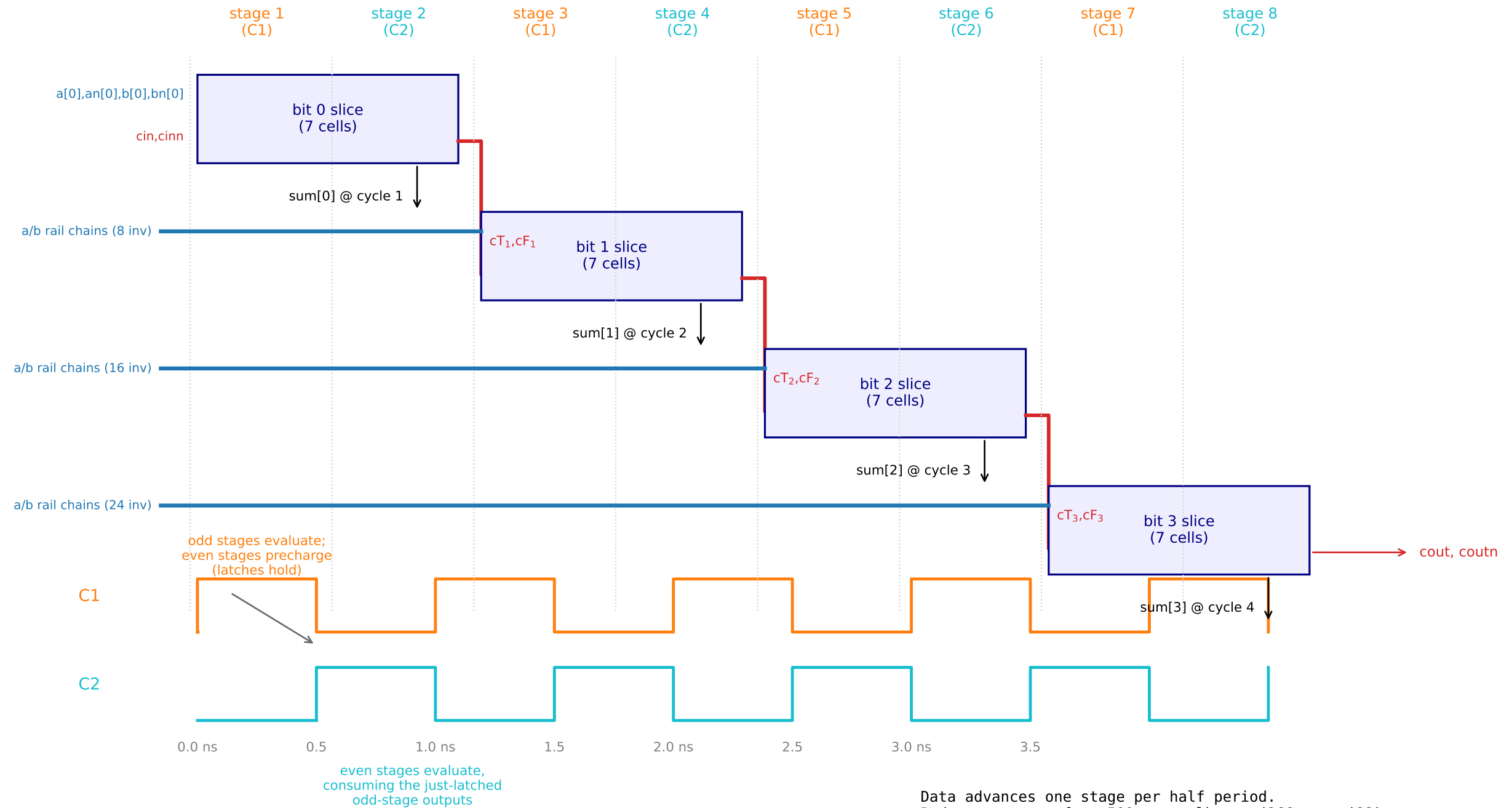
a[i], an[i], b[i], bn[i]	ports	Operand bits, DUAL-RAIL: true + complement versions. The static-CMOS driver (LFSR) supplies both; LSDL cannot make a complement at the same stage (every cell inverts).
p_a{i}_s{k}, q_a{i}_s{k}	chain nets	Physical staging-chain wires: k-th inverter output of the pair rooted at a[i] (p) and an[i] (q). After k inversions p carries a[i] iff k is even — so the PAIR always holds both polarities; only the roles swap.
aT, aF / bT, bF @ stage s	role names	Whichever chain wire carries the true (T) / complement (F) value at stage s. rails() in the generator resolves role → physical wire.
cin, cinn	ports	Carry-in rails = "stage-0 outputs" for bit 0.
cT_i, cF_i	stage 2i	Carry rails INTO bit i — true and complement of c_i, produced by bit i−1 cells U_coT / U_coF (or the cin ports).
cTs, cFs	stage 2i+1	Carry rails restaged one stage through U_cst/U_csf, so they arrive at stage 2i+2 together with x_i. Inverting the OPPOSITE rail keeps polarity: cTs = !cF_i = c_i.
x_i	stage 2i+1	Propagate: x = a⊕b, computed in ONE inverting cell via aoi22(aT,bT,aF,bF) = !(ab + !a!b). Used by sum AND both carry cells.
nx_i	stage 2i+1	Complement propagate !(a⊕b), for the sum XOR.
cT_{i+1}, cF_{i+1}	stage 2i+2	Carry-out rails. cF = !(ab + c·x) directly; cT uses majority self-duality maj(a,b,c) = !maj(!a,!b,!c) and !a⊕!b = x, giving !(!a!b + !c·x) — true carry from one inverting cell, no extra stage.
sum[i]	stage 2i+2 / port	Sum bit: s = x⊕c via aoi22(x,cTs,nx,cFs). Outputs are time-skewed: sum[i] valid 2i+2 half-cycles (= i+1 cycles) after its operands.
cout, coutn	stage 2W / ports	Final carry rails of bit W−1.

How the slices form an adder

Standard full-adder identities, restated for inverting-only cells:
sum_i = a ⊕ b ⊕ c = x ⊕ c (x computed once, shared by sum and carry)
cout_i = a · b + c · (a⊕b) = a · b + c · x (when a≠b the carry propagates; when a=b it is generated)

Each bit needs the carry in BOTH polarities (the sum-XOR and the dual carry cells consume cT and cF), so the carry ripples as a rail pair. Bit i+1 consumes cT/cF_{i+1} at stage 2i+3 — the carry advances one bit per two stages = one bit per full clock cycle. Operands for bit i are delayed to stage 2i by their staging chains, so each slice sees its operands and its carry at the same pipeline depth — a classic systolic (skewed) ripple adder. Throughput: one full W-bit addition completes every cycle; latency of sum[i] is i+1 cycles.

4-bit chaining and C1/C2 two-phase timing



Data advances one stage per half period.
Budget per transfer: 500 ps = clk→out (266 ps aoi22)
+ setup (100 ps) + 134 ps slack [OpenSTA, TT/5V/25C].
The internal latch holds each output through precharge,
which is exactly when the opposite phase reads it.